ONETOWNCITY

Technical Stack & Application Architecture

PROPOSAL FOR A PERFORMANCE-FIRST SAAS REBUILD

DECISION SUMMARY

Build a lean modular SaaS platform: Next.js + TypeScript for the experience, Rust + Axum for the API, PostgreSQL for transactional data, and Cloudflare for edge delivery and media storage.

01 EXPERIENCE	02 APPLICATION	03 DATA & EDGE
SEO-friendly public directory and fast, responsive dashboards.	One modular Rust API; clear domain boundaries; bounded resource use.	PostgreSQL as source of truth; cache public reads at the edge.

TARGET INFRASTRUCTURE

2 VPS instances • 4 vCPU total • 4 GB RAM total • 120 GB SSD total

Design stance: maximize perceived speed and maintainability without adding infrastructure that the initial footprint cannot support.

Proposed stack and rationale

The stack is intentionally opinionated. Each technology has one primary job, and the first release remains a modular monolith rather than a fleet of microservices.

LAYER	PROPOSAL	WHY THIS CHOICE
Web experience	Next.js + React + TypeScript	Server-rendered / statically generated public pages, strong SEO primitives, reusable UI and type-safe frontend code. Keep interactive JavaScript focused on the actions that need it.
UI system	Tailwind CSS + shadcn/ui	Consistent spacing, responsive layouts, accessible primitives and a reusable design system without building every component from zero.
API runtime	Rust + Axum + Tokio	Efficient asynchronous I/O, strong types and predictable resource control. Suitable for a compact API service; Rust raises implementation complexity, so enforce shared patterns and code review.
Database access	SQLx + PostgreSQL migrations	Compile-time query checking where applicable, explicit SQL and reviewed schema migrations. Keep query plans visible to developers.
Primary database	PostgreSQL	Strong relational model for categories, listings, organizations, membership, leads and billing; transactions, constraints, indexes and built-in full-text search.
Public caching / edge	Cloudflare CDN and cache rules	Serve static assets and eligible public content close to users. Define cache keys, TTLs and invalidation explicitly; bypass shared cache for user-specific data.
Media storage	Cloudflare R2 or equivalent object storage	Separate photos and uploads from the VPS disks; deliver images through a CDN and avoid serving large files from the API process.
Reverse proxy / TLS	Caddy or Nginx on API host	HTTPS termination, request size/time limits, compression where useful, security headers and clean routing to the API.
CI / observability	GitHub Actions, tracing, metrics and error alerts	Automated builds/tests, repeatable releases and production evidence for latency, errors, memory, CPU, disk and database behavior.

Deliberate exclusions at launch: Kubernetes, microservices, Kafka, Elasticsearch/OpenSearch and a mandatory Redis cluster. Add these only when measured requirements justify their cost and operational burden.

Application architecture

Separate the user-facing delivery layer from the transactional core. The edge serves cacheable public content; the Rust API authorizes and processes dynamic operations; PostgreSQL remains the system of record.

VISITORS / CRAWLERS
Public discovery, business owners and administrators

↓ HTTPS / DNS / edge routing

CLOUDFLARE EDGE
DNS • TLS • CDN • WAF/rate limits • explicit public cache

FRONTEND RUNTIME
Next.js + TypeScript • SEO pages • owner dashboard UI

↓ Dynamic API requests only; public cache hits stop at the edge

SERVER 1 / API
Caddy/Nginx → Rust + Axum modular API → bounded async work queue

↓ Private network / firewall-restricted database connection

SERVER 2 / DATA
PostgreSQL • constraints • indexed reads • migrations • backups sent off-site

EXTERNAL SERVICES
R2 media • email provider • payment gateway • monitoring

Request flows

A. Public listing view	Browser → edge cache → if miss, render/fetch → API → indexed PostgreSQL query → response cached only when safe.
B. Business owner action	Authenticated UI → API verifies identity, role and tenant → transaction/validation → PostgreSQL → audit event → response.
C. Slow or external work	API validates and records an idempotent job → bounded worker performs email/image/notification task → status and logs captured.

Small-server rule: the worker must be bounded and low priority. Do not run CPU-heavy jobs or unbounded concurrency on the API server. Start without a separate message broker; add one if job volume and delivery guarantees require it.

SaaS domain architecture

The directory is the acquisition surface; the paid SaaS layer gives each business a place to manage its presence and convert views into enquiries. Model ownership and entitlements from the beginning.

DIRECTORY Categories, subcategories, listings, locations, photos	**IDENTITY** Users, sessions, verification, role assignments	**TENANCY** Organizations, memberships, tenant-scoped records
MONETIZATION Plans, subscriptions, entitlements, payment events	**ENGAGEMENT** Enquiries, leads, reviews, offers and notifications	**GOVERNANCE** Moderation, reports, audit history and admin actions

Performance design rules

- Cache public pages and assets at the edge; purge/revalidate when a listing is published, edited or removed.
- Use indexed filters, deterministic sorting and bounded page sizes; prefer cursor pagination for large result sets.
- Avoid N+1 reads; fetch related data intentionally and inspect slow queries with EXPLAIN (ANALYZE, BUFFERS).
- Use PostgreSQL full-text search and suitable trigram matching first. Introduce a dedicated search engine only when relevance, typo tolerance or facets justify it.
- Compress and resize images, lazy-load below-the-fold media and keep frontend JavaScript focused.
- Track P50/P95/P99 latency, error rate, cache-hit rate, CPU, RAM, disk, DB wait time and slow queries.

Tenant isolation and authorization

Every tenant-owned record should have an explicit tenant/organization relationship. The API must derive tenant scope from verified membership, not trust a tenant ID supplied by the browser. Test cross-tenant access on read and write paths; consider PostgreSQL row-level security as defense in depth. Cache keys, object-storage keys and background jobs must also preserve tenant boundaries.

Two-server deployment and operating boundaries

The budgeted infrastructure is a constrained starting point, not a high-availability design. The services are split by role so API and database workloads do not compete for the same memory.

RESOURCE	HOSTING ROLE	OPERATIONAL NOTES
Server 1	2 vCPU / 2 GB RAM API host	Rust API and reverse proxy. One bounded worker process if needed. Keep memory headroom for OS, TLS and logs.
Server 2	2 vCPU / 2 GB RAM Database host	PostgreSQL only. Restrict inbound access to the API server/private network. Tune connections and memory conservatively.
Edge platform	Cloudflare	Frontend runtime, CDN, DNS, TLS and public caching, subject to compatibility testing for the selected Next.js deployment path.
Object storage	R2 or equivalent	Store media outside the VPS; apply authorization before issuing private/signed asset URLs.
Backups	Off-site object storage	Automated encrypted backups, retention policy, restore drills and documented recovery objectives.

Non-negotiable implementation decisions

- Build one API as a modular monolith with clean modules and internal interfaces; do not start with microservices.
- Do not expose PostgreSQL to the public internet. Separate credentials and use least privilege.
- Use reviewed migrations, CI checks, staging smoke tests, structured logs and rollback steps for every release.
- Never cache authenticated dashboard responses in a shared public cache.
- Do not promise uninterrupted availability: either VPS can fail, and the database host is a single point of failure.
- Load-test gradually in staging and publish limits only after repeatable results. A fast language is not a substitute for measured query and cache performance.

Recommended delivery order

1. Preserve and audit existing data/URLs → 2. API + schema foundation → 3. Public directory + cache → 4. Tenant accounts and owner dashboard → 5. Plans, billing and leads → 6. Security, observability, load tests and gradual cutover.

Official implementation references

- Cloudflare Workers: Next.js deployment path | https://developers.cloudflare.com/workers/framework-guides/web-apps/nextjs/
- PostgreSQL: full-text search | https://www.postgresql.org/docs/current/textsearch.html
- OWASP: Multi-Tenant Security Cheat Sheet | https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html
- Axum documentation | https://docs.rs/axum/latest/axum/
- SQLx documentation | https://docs.rs/sqlx/latest/sqlx/

Decision note: This is the proposed target architecture. Validate framework/runtime compatibility and benchmark the representative search and listing flows before locking deployment details.